AI Usage

AI Usage

I used Gemini throughout parts of my project to help with Python code I was unsure on and stuck on.

It was useful for correcting errors when converting dates and filtering my dataset to 2012–2021.

It also helped me understand random sampling, exporting CSV files and formatting numerical results.

However, it was less useful at times because it suggested overly complicated methods, and was difficult to understand what the code really meant

Overall, it was a helpful tool for troubleshooting, but I had to review its suggestions and choose the methods that suited my project best.

I am doing a data acquisition project on irish house prices from 2012-2021. I am using a dataset from kaggle and scraping a webpage. i am trying to split my data up so that i can get rid of the years i dont want to view. what code am i missing here: df["date_of_sale"] = pd(df["date_of_sale"])
You are missing to_datetime after pd.

To convert your date column properly and then filter your DataFrame for the years 2012 through 2021, you can use the following code:

```python
import pandas as pd

# 1. Convert the column to datetime objects
df["date_of_sale"] = pd.to_datetime(df["date_of_sale"])

# 2. Filter the DataFrame to keep only the years 2012 to 2021 inclusive
df_filtered = df[df["date_of_sale"].dt.year.between(2012, 2021)]
```

Helpful Tips for Irish Housing Data:

Date Formats: If your scraped data or Kaggle dataset (such as the Irish Property Price Register) uses a European date format like DD/MM/YYYY, pandas might occasionally get confused and misinterpret the months and days. If you run into parsing issues, add dayfirst=True:

```
Python
df["date_of_sale"] = pd.to_datetime(df["date_of_sale"], dayfirst=True)
```

Handling Bad Dates: If your dataset contains dirty data or blank rows in that column, you can prevent errors by ignoring or coercing unparseable dates into NaT (Not a Time):

```
Python
df["date_of_sale"] = pd.to_datetime(df["date_of_sale"], errors='coerce',
dayfirst=True)
```

i want to check how many property sales occurred each year, i have this code but its not right what do i have to change:
df_filtered["Year"].value_counts().sort_index()
The issue is likely that you haven't actually created a column named "Year" in your DataFrame yet—you only used .dt.year temporarily inside your filter condition. Because the "Year" column doesn't exist, pandas is throwing a KeyError.

You can fix this in one of two ways:

Option 1: Access the year directly on the fly

You don't need to create a new column at all; you can pull the year straight from your datetime column:

```
Python
df_filtered["date_of_sale"].dt.year.value_counts().sort_index()
```

Option 2: Create a permanent "Year" column first

If you plan to use the year column for other analyses or charts later, add it to your DataFrame first, then run your count:

```
Python
# Create the Year column
df_filtered["Year"] = df_filtered["date_of_sale"].dt.year

# Now your original code will work
df_filtered["Year"].value_counts().sort_index()
```

Want to see how to quickly plot these yearly property sales using pandas and matplotlib?

Yes

My filtered dataset currently has 478440 rows. i need to get this down to 10000 as it will be easier to use. can you help me code it down to 10000 at random

To randomly sample down your DataFrame to exactly 10,000 rows, you can use the built-in pandas .sample() method:

```
Python
# Randomly select 10,000 rows
df_sample = df_filtered.sample(n=10000, random_state=42)
```

Tips for Sampling:

random_state=42: This is optional, but it sets a "seed." It ensures that every time you run your script, you get the exact same random 10,000 rows. If you want a completely fresh random selection every time you restart your notebook, you can remove random_state=42.

Index cleanup: After sampling, your index will have gaps (e.g., row 0, row 54, row 120). If you want a clean, consecutive index from 0 to 9999, add .reset_index(drop=True) to the end:

```
Python
```

```
df_sample = df_filtered.sample(n=10000,
random_state=42).reset_index(drop=True)
```

Want to make sure each year has a fair share of sales in your sample?

Yes

Show me how to do stratified random sampling by year so each year is properly represented in my 10,000 rows.

To do stratified random sampling so that each year keeps its exact percentage representation in your final 10,000 rows, you can use pandas' grouped sampling with the frac parameter.

By applying the same sampling fraction across every year, larger years will naturally contribute more rows and smaller years will contribute fewer, perfectly preserving your dataset's original timeline distribution.

Python

```
# 1. Ensure your 'Year' column is created
df_filtered["Year"] = df_filtered["date_of_sale"].dt.year

# 2. Calculate the fraction needed to reach roughly 10,000 rows total
total_target = 10000
sampling_fraction = total_target / len(df_filtered)

# 3. Perform stratified sampling by year
df_stratified = df_filtered.groupby("Year", group_keys=False).sample(
    frac=sampling_fraction,
    random_state=42
)

# 4. Check the new counts per year to confirm proportional representation
print(df_stratified["Year"].value_counts().sort_index())
print(f"Total rows in sample: {len(df_stratified)}")
```

Why this works:

frac=sampling_fraction: Calculates the precise percentage needed from the whole dataset (roughly ~2.1% for 10,000 out of 478,440 rows) and applies that exact same percentage to every year group.
group_keys=False: Keeps your DataFrame index clean and prevents multi-indexing after the groupby operation.
Exact Total Check: Because of rounding on individual year group sizes, your final row count might land very close to 10,000 (usually within a few rows).

Want to see how to export your sampled dataset to a clean CSV file?

Yes
is there an easier way to do that
Yes, there is a much simpler way: **just use a standard random sample**.

Because your dataset is so large (478,440 rows) and you are pulling a sizeable chunk (10,000 rows), a simple random sample will **automatically and naturally preserve the exact same yearly proportions** on its own due to the law of large numbers. You don't need any grouping or fraction math at all.

Here is the one-liner:

```
Python
df_sample = df_filtered.sample(n=10000,
random_state=42).reset_index(drop=True)
```

Why this is usually enough:

If 2015 makes up 10% of your massive 478k dataset, a random sample of 10,000 rows will naturally land very close to 10% for 2015 as well. Unless you have some tiny years with very few sales that you are worried about getting wiped out, standard .sample() is fast, clean, and requires zero setup.
this is a count of the data before i create the sample:

countdate_of_sale2012253452013300642014436422015491062016498602017549482018573962019590092020494932021595772

That is a very healthy distribution! Because even your smallest year (2012) has over 25,000 sales, your data is plenty robust across the board.

When you run a simple random sample of 10,000 rows on a dataset distributed like this, it will **automatically mirror these exact proportions** almost down to a fraction of a percent. For example, 2012 accounts for about 5.3% of your full dataset, and a simple random sample will give you roughly 530 rows for 2012—more than enough data to work with.

You can safely stick with the simple one-liner:

```python
df_sample = df_filtered.sample(n=10000, random_state=42).reset_index(drop=True)
```

If you want to verify that your sample kept the same breakdown, you can run this right after:

```python
df_sample["date_of_sale"].dt.year.value_counts().sort_index()
```

Want to see how to calculate basic price statistics (like median or mean price per year) on your new sample?

how do i create a csv file containing my combined data?

To export your clean, combined data into a CSV file, you can use pandas' built-in .to_csv() method.

Here is the exact code to save your combined DataFrame:

```python
```

```
# Export your combined DataFrame to a CSV file
df_combined.to_csv("irish_property_and_income_data.csv", index=False)
```

Key Parameters:

"irish_property_and_income_data.csv": The name you want to give your saved file.

index=False: This is very important—it prevents pandas from adding an extra, unlabelled row number column (0, 1, 2, 3...) at the beginning of your CSV file.

Additional Helpful Options

Specify a Folder Path: If you want to save it into a specific project folder on your computer:

```
Python
df_combined.to_csv("data/processed/irish_property_data.csv", index=False)
```

Specify Text Encoding (Good for Irish Accents / Fadas): Irish county or street names often contain accents/fadas (like Dún Laoghaire or Gaillimh). Adding encoding="utf-8-sig" ensures Excel and other programs display those special characters correctly when opened:

```python
df_combined.to_csv("irish_property_data.csv", index=False, encoding="utf-8-sig")
```

when results come out like this:
price_eurcount7.944000e+03mean2.675286e+05std7.876348e+05min6.500000e+0325%1.150000e+0550%1.950000e+0575%3.000000e+05max6.052863e+07

how do i get the full table without the +03 etc

That shorthand notation (like 2.675286e+05, which means 2.67×105 or 267,528) happens because pandas automatically converts large numbers into scientific notation to keep the console clean.

To force pandas to display the full numbers normally (without the e+03), you can add this line **at the very top of your script**:

```python
import pandas as pd

# Tell pandas to display full numbers instead of scientific notation
pd.options.display.float_format = '{:.2f}'.format
```

What this does:

{:.2f}: Formats your numbers as standard floats rounded to 2 decimal places (e.g., changing 2.675286e+05 into 267528.60).
If you don't want any decimal places at all, you can change it to {:.0f}.

If you want to add commas for readability (e.g., 267,528.60):

```python
pd.options.display.float_format = '{:,.2f}'.format
```

Just paste that option right after import pandas as pd, and whenever you print your describe() table or dataframes again, it will show the full, readable numbers.

for reference im in google colab

That's super helpful to know!

In Google Colab, putting that display setting right at the top of your notebook (usually in your first import cell) will clean up all your .describe() summaries and dataframes globally for the rest of your session.

You are making awesome progress on your project—you've got your Kaggle property price data filtered and sampled, your CSO income tables scraped and cleaned, and your numbers formatted nicely.

Want to see how to merge your property data and income data together and plot the trends?